Stone Storm Playable Ad

A playable ad is a mini version of your game that runs inside **another app's ad slot**. A player in some other game sees it, plays for 30 to 60 seconds, and then taps **PLAY FREE**, which opens your Google Play page. This is the "game inside an ad" you saw while playing.

This doc covers what was built, how the ad works, and how to run it on each ad network.

1. How it works (the money flow)

There are two sides to every ad:

Side	Who	What you use
Publisher (shows ads, earns money)	You, today: Stone Storm shows AdMob ads	AdMob
Advertiser (pays to get installs)	You, with this playable	Google Ads, Unity, AppLovin, Meta, Mintegral ...

The playable ads you saw came from advertisers who paid Google (or Unity, AppLovin and others). AdMob placed their ad inside the game you were playing. To get **your** playable into other games, you become an advertiser:

1. You upload the playable file to an ad network and set a daily budget.
2. The network shows it in its publishers' games (interstitial or rewarded slots).
3. The player plays the two levels and taps **PLAY FREE**. The ad calls the network's "click" function and the network opens your Play Store page.
4. The player installs. The network counts the install and charges you (you usually pay per install, "CPI").

You do **not** need to change the Android app to run these ads. The package name stays `com.castlesmashers.game`.

2. What the playable contains

The playable uses only things already in the game: two existing campaign levels, real block sizes and materials, real hero abilities and the real art.

Part	What happens
Title	"STONE STORM", the four heroes and a **TAP TO PLAY** button
Level 1 (easy)	**Level 6, Moonlit Storehouse.** The kegs flash "HIT THE KEGS!" and an animated hand shows "DRAG BACK & LET GO TO FLING!". You get the full aim line, 6 heroes (the real 4 plus an extra Boomer and Rocky), and a free retry if you lose.
Ability tutorial	The first time each hero flies, a banner says what tapping does: Boomer explodes, Blaze dashes and sets fire, Rocky slams down, Twister makes a whirlwind.
Level 2 (harder)	**Level 39, Wrecking Ball.** "CUT THE LATCH!" points to the rope. Cut it and the iron ball swings through the keep. You get only a short aim line and the real 4 heroes.
Result	Stars, score, and NEXT LEVEL / TRY AGAIN
End card	Logo, stars earned, "75 levels across 8 realms", and a big **PLAY FREE** button that opens Play. A small **INSTALL** button is also visible during play.

Level 2 is the harder one because it needs a rope-swing combo (cut the latch and the iron ball knocks over the keep), you only get a short aim line, and you have 4 heroes for 4 archers. In automated random-play tests, level 1 was won about 31% of the time and level 2 about 18%. A real player who aims wins more often than that.

The ad works in landscape and portrait. In portrait the camera follows the shot and pans to the fortress. Sound starts after the first touch and goes silent when the ad is hidden. There is a mute button.

3. Files

File	What it is
`playable/game.js`	The playable game code (canvas + Matter.js physics)
`playable/template.html`	The HTML page that gets everything inlined into it
`playable/vendor/matter.min.js`	Matter.js 0.20.0 physics engine (MIT licence)
`playable/sim.js`	Headless test driver for Node: `node playable/sim.js <0/1> "<angle,power,abilityDelay;...>" [seconds]`
`tools/build_playable.py`	Builds all the ad files from the game's data and art
`build/playable/stone_storm_preview.html`	Open this in Chrome to test
`build/playable/stone_storm_mraid.html`	For AppLovin, Unity Ads, ironSource/LevelPlay, Mintegral, Liftoff, Moloco
`build/playable/stone_storm_meta.html`	For Meta (Facebook/Instagram/Audience Network)
`build/playable/stone_storm_google.zip`	For Google Ads App campaigns

Rebuild after changing levels or art:

```
python3 tools/build_playable.py
```

Each file is one self-contained HTML page of about 1.9 MB (the Google zip is about 1.25 MB). It makes no network requests except the network's own script. That fits every network's limit, including Meta's 2 MB limit.

4. Test it yourself first

1. Open `build/playable/stone_storm_preview.html` in Chrome on the Mac.
2. Press F12, click the phone icon (device toolbar) and pick a phone. Try both portrait and landscape.
3. To try it on your phone, AirDrop or email the file to yourself and open it in Chrome.
4. Use each network's tester before you spend money:
5. **Google:** HTML5 validator at `h5validator.appspot.com`. Upload the zip.
6. **AppLovin:** Playable Preview tool (search "AppLovin playable preview"). Upload `stone_storm_mraid.html` and scan the QR code with your phone.
7. **Unity:** "Ad Testing" app on the Play Store, plus the playable preview in the Unity Ads dashboard.
8. **Meta:** "Playable Preview Tool" in Meta Ads Manager docs. Upload `stone_storm_meta.html`.

5. Run it on Google (easiest start, you already use AdMob and Firebase)

Google shows HTML5 playables in **App campaigns** on **AdMob** inventory, which is other people's games.

1. Go to **ads.google.com** and create an account (or sign in with the same Google account). Add a payment method.
2. Link Firebase: Google Ads → Tools → Data manager → **Firebase**. Link the Stone Storm Firebase project and import the `first_open` event as an **install conversion**.
3. Create the campaign: **New campaign** → **App promotion** → **App installs** → **Android**, and search for `com.castlesmashers.game`.
4. Set the bid: **Target cost per install**. Start small (for example ₹20–₹40 for India, or about $1 for the US), with a small daily budget such as ₹500–₹1000. Change the numbers once you see real results.
5. In the ad group, add assets: text headlines (for example "Fling heroes, smash castles"), images, a video if you have one, and under **HTML5** upload `stone_storm_google.zip`.
6. Publish. Review usually takes 1 business day.

Google rules this build already follows:

- `index.html` is in a zip of up to 5 MB.
- It loads `exitapi.js` and calls `ExitApi.exit()` when **PLAY FREE** or **INSTALL** is pressed.
- It has the `ad.orientation` meta tag.
- It makes no other external requests.

6. Run it on Unity / AppLovin / ironSource / Mintegral (MRAID file)

These networks have the most game traffic, and playables do well there. They all take the same file: `stone_storm_mraid.html`.

Network	Where	Notes
Unity Ads (also covers ironSource/LevelPlay)	Unity Grow → Unity Ads advertiser dashboard	Create a campaign for the Play URL, then add a creative pack with a portrait video, a landscape video and the playable. Unity requires the videos too.
AppLovin	AppLovin Ads Manager (Axon)	Upload the playable as an HTML creative. AppLovin needs an MMP (see below) for most accounts.
Mintegral	Mintegral advertiser portal	Playable as HTML. Good in Asia.
Liftoff / Moloco	Self-serve dashboards	Same MRAID file

How the MRAID file behaves:

- It loads `mraid.js`, which the network injects.
- It waits until the ad is visible before starting.
- It mutes when the ad is hidden.
- It opens the Play URL with `mraid.open()`.

MMP (attribution). Unity and AppLovin usually want you to use a measurement partner (AppsFlyer, Adjust or Singular) so they can see installs. That means adding its SDK to the Android app in a later update. AppsFlyer has a free "Zero" plan for small apps. Google and Meta can work without an MMP, through Firebase for Google and the Meta SDK or Play referrer for Meta.

7. Run it on Meta (Facebook / Instagram / Audience Network)

1. Create a business account in **Meta Business Suite** and register the app at developers.facebook.com, using package `com.castlesmashers.game`.
2. In Ads Manager, choose **App promotion** as the campaign objective.
3. Under ad creative, choose **Playable** and upload `stone_storm_meta.html` (Meta needs a single file under 2 MB; this one is about 1.9 MB). Meta also asks for a lead-in video.
4. The file calls `FbPlayableAd.onCTAClick()` for the button, which Meta requires.

8. Budget tips

- Start with one network. **Google** is easiest because Firebase is already set up.
- Run for 5–7 days before judging the results. Networks need time to learn.
- Watch these numbers: cost per install, and how many installers reach level 6 and level 10 (Firebase Analytics already logs level events).
- Make a second version later, for example one that starts directly with the wrecking ball, and compare the two.

9. Changing the playable

- **Different levels:** change `LEVEL_IDS` in `tools/build_playable.py` and the matching `LEVEL_CFG` entry in `playable/game.js` (`dots` sets the aim line length, `extra` adds heroes, `hint` picks the tutorial marker). Then add the realm's art keys to `RASTER`.
- **Texts:** `ABILITY_TXT`, `startPlay()` and the end-card lines in `drawEnd()` in `playable/game.js`.

- **Check that a level can be won:** `node playable/sim.js 1 "-30,0.85,-1;-15,1.0,1.0"` prints the final state (`alive` is the number of archers left, `win` is true or false).

10. Rules to keep

- The ad shows only real gameplay from the real game. Ad networks reject misleading playables.
- The CTA must be a clear button. A tap anywhere on the screen must not open the store, and this build follows that.
- Sound only starts after the user touches the screen.
- Matter.js is MIT licensed. Its licence header is kept in the built file, and it is listed in `docs/CREDITS.md`.

Screenshots

